一. Sophus 库

官网：

https://github.com/strasdat/Sophus

安装：

git clone http://github.com/strasdat/Sophus.git

git checkout a621ff

mkdir build

cd build

cmake ..

make

二. SO(3)和 SE(3)的运算

完整代码：https://github.com/deng-ke/slam-base/tree/master/SophusSOSE。

2.1 构造李群 SO(3)

```
/*****************************************************************************
 *不同方式构造李群 SO(3)
 *****************************************************************************/
    /*用旋转矩阵构造（沿 Z 轴转 90 度）*/
    Eigen::Matrix3d R = Eigen::AngleAxisd(M_PI/2, Eigen::Vector3d(0,0,1)).toRotationMatrix();
    Sophus::SO3 SO3_R(R);
    /*用旋转向量构造*/
    Sophus::SO3 SO3_v( 0, 0, M_PI/2 );
    /*用四元数构造*/
    Eigen::Quaterniond q(R);
    Sophus::SO3 SO3_q( q );
    cout<<"SO(3) from matrix: "<<SO3_R<<endl;
    cout<<"SO(3) from vector: "<<SO3_v<<endl;
    cout<<"SO(3) from quaternion :"<<SO3_q<<endl;
```

```
SO(3) from matrix:      0      0 1.5708

SO(3) from vector:      0      0 1.5708

SO(3) from quaternion :     0      0 1.5708
```

2.2 李代数 so(3)的运算

```
/*****************************************************************************
 *李代数 so(3)的运算
 *****************************************************************************/
    // 使用对数映射获得它的李代数
    Eigen::Vector3d so3 = SO3_R.log();
    cout<<"so3 = "<<so3.transpose()<<endl;
    // hat 为向量到反对称矩阵
    cout<<"so3 hat=\n"<<Sophus::SO3::hat(so3)<<endl;
    // 相对的，vee 为反对称到向量
    cout<<"so3 hat vee= "<<Sophus::SO3::vee( Sophus::SO3::hat(so3) ).transpose()<<endl; // transpose 纯粹是为了输出美观一些
    // 增量扰动模型的更新
    Eigen::Vector3d update_so3(1e-4, 0, 0); //假设更新量为这么多
    Sophus::SO3 SO3_updated = Sophus::SO3::exp(update_so3)*SO3_R;
    cout<<"SO3 updated = "<<SO3_updated<<endl;
```

```
so3 =      0      0 1.5708
so3 hat=
      0 -1.5708       0
 1.5708       0      -0
     -0       0       0
so3 hat vee=      0      0 1.5708
SO3 updated =  7.85398e-05 -7.85398e-05       1.5708
```

2.3 构造李群 SE(3)

```
/*****************************************************************************
 *不同方式构造李群 SE(3)
 *****************************************************************************/
    // 对 SE(3)操作大同小异
    Eigen::Vector3d t(1,0,0);                 // 沿 X 轴平移 1
    Sophus::SE3 SE3_Rt(R, t);                 // 从 R,t 构造 SE(3)
    Sophus::SE3 SE3_qt(q,t);                  // 从 q,t 构造 SE(3)
    cout<<"SE3 from R,t= "<<endl<<SE3_Rt<<endl;
    cout<<"SE3 from q,t= "<<endl<<SE3_qt<<endl;
```

```
SE3 from R,t=
     0      0 1.5708
1 0 0

SE3 from q,t=
     0      0 1.5708
1 0 0
```

2.4 李代数 s(3)的运算

```
/**********************************************************************************
 *李代数 se(3)的运算
 **********************************************************************************/
    // 李代数 se(3) 是一个六维向量，方便起见先 typedef 一下
    typedef Eigen::Matrix<double,6,1> Vector6d;
    Vector6d se3 = SE3_Rt.log();
    cout<<"se3 = "<<se3.transpose()<<endl;
    // 观察输出，会发现在 Sophus 中，se(3)的平移在前，旋转在后.
    // 同样的，有 hat 和 vee 两个算符
    cout<<"se3 hat = "<<endl<<Sophus::SE3::hat(se3)<<endl;
    cout<<"se3 hat vee = "<<Sophus::SE3::vee( Sophus::SE3::hat(se3) ).transpose()<<endl;

    // 最后，演示一下更新
    Vector6d update_se3; //更新量
    update_se3.setZero();
    update_se3(0,0) = 1e-4d;
    Sophus::SE3 SE3_updated = Sophus::SE3::exp(update_se3)*SE3_Rt;
    cout<<"SE3 updated = "<<endl<<SE3_updated.matrix()<<endl;
```

```
se3 =  0.785398 -0.785398         0         0         0    1.5708
se3 hat =
        0   -1.5708         0  0.785398
   1.5708         0        -0 -0.785398
       -0         0         0         0
        0         0         0         0
se3 hat vee =  0.785398 -0.785398         0         0         0    1.5708
SE3 updated =
2.22045e-16          -1           0      1.0001
          1 2.22045e-16           0           0
          0           0           1           0
          0           0           0           1
```